

Technical Report: X Investor Copilot Codebase

This report reflects the current repository state at:
'C:\Users\zacka\x_investor_copilot'

Primary references:

- 'README.md'
- 'docker-compose.yml'
- 'infra/docker-compose.yml'

1. System Overview

This is a monorepo MVP implementing a B2C "save from X + chat over saved corpus" workflow with three runtime components:

1. 'apps/api' (FastAPI + Postgres + pgvector)
2. 'apps/web' (Next.js App Router + Clerk auth + dashboard)
3. 'apps/extension' (Chrome MV3 extension for capture + side panel chat)

The architecture is intentionally simple:

- Postgres is both system-of-record and vector store.
- Ingestion and embedding are synchronous during request handling.
- Extension authentication uses PAT.
- Web authentication uses Clerk; web server forwards Clerk identity headers to API.

2. Core Design Decisions

Decision A: Separate auth planes (Clerk for web, PAT for extension)

- Implemented in API deps and web forwarding:
 - 'apps/api/app/api/deps.py'
 - 'apps/web/src/lib/server-api.ts'
- Why: low-friction extension auth for MVP while preserving standard web auth UX.
- Tradeoff: dual auth model increases complexity and introduces trust-boundary issues.

Decision B: Postgres + pgvector as single backend store

- 'apps/api/app/db/models.py'
- 'infra/db/init.sql'
- Why: minimal infra and strong transactional semantics across metadata + vectors.
- Tradeoff: retrieval performance tuning and horizontal scale are deferred.

Decision C: Deterministic local embeddings instead of provider embeddings

- 'apps/api/app/services/embeddings.py'
- Why: zero external dependency/cost for MVP.
- Tradeoff: retrieval quality is materially lower than semantic embeddings.

Decision D: DOM-only, user-initiated capture in extension

- 'apps/extension/content-script.js'
- Why: explicit compliance with no background crawling / no interception.
- Tradeoff: capture completeness depends on what is currently rendered and expanded.

Decision E: Source-grounded response format with explicit Unknown / Speculation fallback

- 'apps/api/app/services/rag.py'
- 'apps/api/app/services/prompts.py'
- Why: enforce attribution and reduce hallucination risk.
- Tradeoff: response quality is heuristic (no real LLM synthesis currently).

3. Monorepo and Runtime Topology

Top-level workspace config:

- 'pnpm-workspace.yaml'
- 'package.json'

Compose runtime:

- 'docker-compose.yml'

Runtime graph:

1. 'db' uses 'pgvector/pgvector:pg16'
2. 'api' depends on 'db' healthcheck and runs migrations before serving
3. 'web' depends on 'api'
4. Extension calls 'api' directly (localhost) and does not go through 'web'

4. Backend Deep Dive (FastAPI)

Primary entry:

- 'apps/api/app/main.py'
- 'apps/api/app/api/__init__.py'

Configuration:

- 'apps/api/app/core/config.py'

Database session:

- 'apps/api/app/db/session.py'

Libraries used:

- FastAPI / Starlette
- SQLAlchemy 2.x
- Alembic
- psycopg3
- pgvector SQLAlchemy integration
- pydantic-settings

5. API Authentication and Identity Control Flow

PAT auth path

1. Parse 'Authorization: Bearer ...'
2. HMAC-hash token with 'TOKEN_PEPPER'
3. Join 'api_tokens -> users'
4. Reject if missing/revoked
5. Update 'last_used_at'

Code:

- 'apps/api/app/api/deps.py'
- 'apps/api/app/core/security.py'

Clerk-header auth path

1. Read 'x-clerk-user-id' and optional email
2. Upsert 'users' by 'clerk_user_id'

Code:

- 'apps/api/app/api/deps.py'

Important behavior:

- '/v1/chat' and '/v1/library/*' accept PAT or Clerk headers ('get_any_authenticated_user').
- '/v1/tokens' uses Clerk-header auth ('get_current_clerk_user').

6. Database Design

Models:

- 'apps/api/app/db/models.py'
- Migration: 'apps/api/alembic/versions/20260227_0001_init_schema.py'

Tables and intent:

1. 'users'
 - Canonical user identity, keyed by UUID.
 - 'clerk_user_id' unique index.
2. 'api_tokens'
 - PAT registry, stores 'token_hash', not plaintext.
 - 'token_fingerprint' for display.
 - revocation and last-used metadata.
3. 'x_items'
 - Atomic tweet capture units.
 - Unique '(user_id, tweet_id)' dedupe boundary.
 - JSON fields for quoted/raw payload preservation.
4. 'x_threads'
 - Thread-level capture metadata.
 - Includes partial-capture flags.
5. 'x_thread_items'
 - Junction table for thread->item mapping.
6. 'chunks'
 - Vector-searchable chunk records.
 - 'source_type' in practice: 'x_item' or 'x_thread'.
 - 'embedding' vector(256), 'metadata_json' stores citation metadata.

Critical indexing choices:

- 'users.clerk_user_id' unique
- 'api_tokens.token_hash' unique
- 'x_items(user_id, tweet_id)' unique
- 'chunks' indexed by 'user_id', 'source_type', 'source_id'

7. Endpoint Behavior and Control Flow

Route files:

- 'apps/api/app/api/routes/tokens.py'
- 'apps/api/app/api/routes/ingest.py'
- 'apps/api/app/api/routes/chat.py'
- 'apps/api/app/api/routes/library.py'

Token lifecycle

1. 'POST /v1/tokens' creates plaintext token once and stores hash.
2. 'GET /v1/tokens' lists active/revoked token metadata.
3. 'DELETE /v1/tokens/{id}' sets 'revoked_at'.

Ingest lifecycle ('POST /v1/ingest/x')

1. Request-level dedupe by tweet_id set.
2. Normalize tweet id from payload or URL.

3. Upsert 'x_items' by '(user_id, tweet_id)'.
4. Create one per-item chunk if not already present.
5. If capture_type='thread':
 - create 'x_threads' row
 - link items via 'x_thread_items'
 - create one macro thread chunk
6. Commit once at end.

Chat lifecycle ('POST /v1/chat')

1. Authenticate user.
2. Embed query text via local deterministic embedding.
3. Retrieve nearest chunks by cosine distance.
4. Apply optional filters and optional thread scope.
5. Build structured answer + citations with heuristic categorization.

Library reads

- List items
- List threads with aggregate 'item_count'
- Thread detail with ordered items

8. RAG/Embedding Strategy

Embedding

- Hash-based bag-of-token projection into 256-dim normalized vector.
- No external model call.
- 'apps/api/app/services/embeddings.py'

Chunking

- Per tweet chunk includes author, URL, text, optional quoted tweet text.
- Thread macro chunk concatenates ordered tweets with size cap.
- 'apps/api/app/services/ingest.py'

Retrieval

- SQL-level cosine distance over pgvector.
- Oversampling ('top_k * 4', min 20) then metadata filter pass in Python.
- 'apps/api/app/services/rag.py'

Answer synthesis

- No model call currently.
- Heuristic classification into Facts/Opinions/Forecasts/Bull/Bear/Uncertainties.
- "Unknown / Speculation" fallback when unsupported.
- Prompt constant exists but is not wired into generation runtime:
'apps/api/app/services/prompts.py'

9. Web App Deep Dive (Next.js + Clerk)

Core files:

- 'apps/web/middleware.ts'
- 'apps/web/src/app/layout.tsx'
- 'apps/web/src/lib/server-api.ts'

Auth model:

- Clerk middleware protects '/app/*' and '/api/*'.
- Next server-side route handlers call FastAPI and forward 'x-clerk-user-id' headers.
- This creates a thin BFF pattern for authenticated dashboard operations.

UI route map:

- '/' landing
- '/app/library'
- '/app/chat'
- '/app/settings/tokens'
- '/app/threads/[id]'
- '/sign-in/[[...sign-in]]'
- '/sign-up/[[...sign-up]]'

Internal API proxy routes:

- 'apps/web/src/app/api/tokens/route.ts'
- 'apps/web/src/app/api/tokens/[id]/route.ts'
- 'apps/web/src/app/api/chat/route.ts'
- 'apps/web/src/app/api/library/items/route.ts'
- 'apps/web/src/app/api/library/threads/route.ts'
- 'apps/web/src/app/api/library/threads/[id]/route.ts'

Notable implementation detail:

- If Clerk key is absent, root layout renders without 'ClerkProvider' fallback; sign-in/up pages also fallback text.
- This helps local builds but does not replace real auth in runtime.

10. Extension Deep Dive (MV3)

Core files:

- 'apps/extension/manifest.json'
- 'apps/extension/background.js'
- 'apps/extension/content-script.js'
- 'apps/extension/options.js'
- 'apps/extension/sidepanel.js'

Control flow:

1. User clicks extension-injected button on x.com.
2. Content script captures visible DOM tweet data only.
3. Content script sends message to service worker ('INGEST_X').
4. Service worker reads PAT from 'chrome.storage.sync' and calls FastAPI.
5. For chat, sidepanel sends 'CHAT' message to worker and renders response/citations.

Capture model:

- Save Tweet captures one current tweet.
- Save Thread captures all visible tweet articles after bounded expansion attempts.
- Partial capture inferred if "show more replies/show replies" still present.
- Optional quoted tweet extraction is best-effort.

11. Infrastructure and Build

Compose files:

- 'docker-compose.yml'
- 'infra/docker-compose.yml'

Build decisions:

- API container installs via 'uv pip install --system .'.
- Web container uses multi-stage pnpm workspace-aware Dockerfile.
- Root '.dockerignore' avoids host 'node_modules', '.venv', '.next' contamination.

Note:

- README references '.env.example', but file is currently missing from root in this workspace snapshot.

This is a documentation/setup gap.

12. Security and Trust Boundaries

Critical points for senior review:

1. Header trust model for Clerk identity in FastAPI.
 - '/v1/tokens' trusts 'x-clerk-user-id' headers and does not validate Clerk JWT.
 - Intended MVP shortcut, but directly calling API can spoof identity if endpoint is exposed.
2. PAT storage in extension.
 - PAT stored in 'chrome.storage.sync' plaintext form.
 - Sync storage improves UX but expands token exposure surface.
3. CORS policy is permissive.
 - API sets 'allow_origins=["*"]' and 'allow_credentials=True'.
 - Should be tightened for production origins and extension context.
4. Token lifecycle limitations.
 - No token expiry, no scoped permissions, no rotate endpoint.
 - Revocation supported, hashing is HMAC(pepper, token) which is good baseline for random tokens.

13. Performance and Scalability Characteristics

Current behavior:

- Ingest path is synchronous and includes embedding + DB writes.
- Chat retrieval is single-query vector search + Python filtering.
- Token 'last_used_at' updates commit per PAT-authenticated call.

Implications:

- Good MVP simplicity and consistency.
- Throughput bottlenecks likely at DB for high ingest/chat concurrency.
- No async queueing, no caching, no batch embedding pipeline.

14. Gaps, Risks, and Engineering Priorities

High-priority technical debt:

1. Add real Clerk JWT verification in API for web-authenticated endpoints.
2. Introduce production embedding + LLM provider integration.
3. Harden CORS and token handling.
4. Implement thread dedupe/versioning (currently new 'x_threads' row per thread capture).
5. Add automated tests (API unit/integration + extension capture harness + web e2e).
6. Restore/add '.env.example' to match onboarding docs.

Medium-priority:

1. Replace heuristic chat synthesis with grounded prompt + LLM call.
2. Add observability (structured logs, request IDs, metrics).
3. Add paging/sorting/filter controls in web UI and extension side panel.
4. Improve capture robustness against x.com DOM changes.

Low-priority:

1. UI polish issues (example: thread back-link displays '?' due character issue).
2. Consolidate duplicate compose definitions if not both needed.

15. What a Senior Engineer Should Know Before Modifying

1. Most business logic lives in FastAPI service modules and ingest/chat routes, not in web.

2. Web layer is mostly a secure proxy + dashboard UI; extension talks directly to API.
3. Authentication is intentionally split and currently asymmetric in security guarantees.
4. Retrieval quality limitations are architectural, not just prompt tuning issues.
5. Database schema is good MVP foundation but not yet optimized for large-scale RAG workloads.
6. The fastest leverage improvements are auth hardening, model integration, and test coverage.